**MPOWERED**

# Development conventions

These conventions help us keep the app consistent and make each other's work easier to review. Follow the patterns in the files you are changing, and check `package.json` before using examples from framework documentation.

## 1. Keep changes focused

Read the relevant code and agreed requirements before starting. Keep each change to one task, and leave unrelated code and formatting alone. If the expected behaviour is unclear, check with the team before building it. Add shared components or extra layers when there is a real need for them.

## 2. Use consistent names and formatting

Use PascalCase for components, such as `YearOfBirthScreen.tsx`, and lowercase route names, such as `year-of-birth.tsx`. Start hook names with `use` and end context names with `Context`. Give assets descriptive, kebab-case names.

Use default exports for routes and main screen components, and named exports for hooks, types, data and styles. Group imports in this order: external packages, `@/` project imports, then relative sibling imports, with a blank line between groups. Use two-space indentation, single quotes, semicolons and trailing commas in multiline code.

## 3. Keep navigation and state simple

Use Expo Router and keep navigation in route files where possible. Pass callbacks such as `onBack` and `onContinue` to screen components. Keep route parameters small and typed, and fix type errors instead of bypassing them with broad casts.

Keep input state local unless another screen needs it; use feature context for values shared across routes. Calculate values such as `canContinue` from existing state. Validate input before continuing, disable unavailable actions, and clean up timers and effects.

## 4. Make screens usable on the target device

Use React Native components and follow the app's existing colours, spacing and typography. Respect safe areas, keep inputs visible when the keyboard opens, and allow scrolling on smaller screens. Limit content width on tablets where it improves readability.

Give controls clear accessibility roles, labels and selected or disabled states. Make tap targets at least 44 by 44 points where possible. Check that users can tell when an action is loading, has failed or has no results, and use accessible announcements for status changes where needed.

## 5. Handle personal data carefully

Use made-up data for testing and screenshots. Keep names, phone numbers, verification codes and health records out of source control, logs and issue descriptions. Keep credentials out of source files and use the project's approved configuration and storage approach.

Check with the team before adding a new way to store or share sensitive data. Privacy, consent, recording and support-person access must follow the agreed requirements. Make any unfinished or simulated behaviour clear in the pull request.

## 6. Make changes easy to review

Start from an up-to-date main branch and create a short-lived branch for one task. Use a descriptive name, such as `fix/splash-small-screen`. Keep commits focused and describe what changed. Open a pull request into main with a short explanation, the checks you ran and any remaining limitations.

Discuss changes to shared dependencies, routes or app configuration with the team. Avoid force-pushing branches others are using. When dependencies change, commit `package.json` and `package-lock.json` together. Keep build output, local environment files and credentials out of commits.

## 7. Check the flow before merging

Run `npx tsc --noEmit` and the relevant checks available in the project. Then try the changed flow from its normal entry point on the target device or simulator. A browser preview is useful, but native behaviour still needs a native check.

Check forward and Back navigation, invalid or empty input, disabled buttons, keyboard behaviour and a fresh reload. Try skip actions, timers, loading and error states where they apply. For layout changes, check a small phone and an iPad, and review the screen's accessibility labels and states.

Before requesting review, read the diff once more. The change should meet the agreed requirement, include only relevant files and be clear about what was actually tested.